TD CoCiNum :
Circuit combinatoire en VHDL

Objectifs pédagogiques de l'activité

- Connaître les principales étapes permettant de réaliser un circuit sur un FPGA.
- Savoir utiliser l'outil `Vivado` pour la synthèse et la simulation d'un circuit logique.
- Décrire un circuit logique combinatoire dans le langage de description matériel VHDL.

Présentation de l'activité

L'objectif de l'activité est de concevoir le circuit logique permettant de contrôler un afficheur 7 segments afin d'afficher un symbole hexadécimal en fonction de l'état de 4 interrupteurs (*switchs*). La correspondance entre l'état des interrupteurs et le symbole à afficher est représenté sur la Figure 1

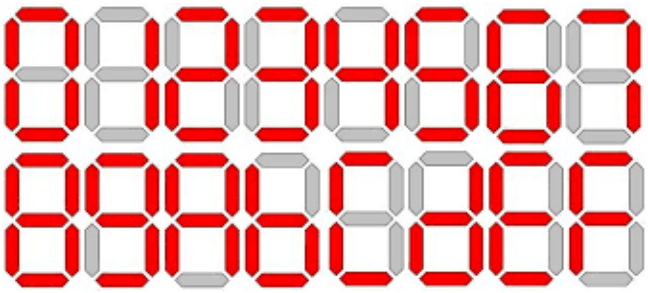

FIGURE 1 – Correspondance entre les interrupteurs et l'affichage du 7 segments

Dans la suite des améliorations seront apportées :
— Détecteur de parité
— Détecteur de nombre premier
— Filtrage de l'affichage

Préparation de l'environnement de travail

L'activité se déroule sous le système d'exploitation Linux : `Ubuntu`.

Téléchargement des fichiers de l'activité

Les fichiers de cette activité et des suivantes sont disponibles en ligne dans un dépôt de code source public.

→ Ouvrez un terminal et exécutez les commandes suivantes :

```
$ cd $HOME
$ git clone https://github.com/ESEO-Tech/CoCiNum.git
$ cd CoCiNum
```

Création du projet Vivado

Vivado est un environnement de développement pour les FPGA du constructeur Xilinx. Nous l'utiliserons pour effectuer les tâches suivantes : éditer des descriptions de circuits en VHDL, simuler leur fonctionnement, les synthétiser, configurer une carte à FPGA.

→ Démarrez Vivado à l'aide de la commande suivante :

```
$ ./scripts/vivado
```

→ Créez un nouveau projet (`Quick Start` → `Create Project`) en renseignant les informations suivantes :

Page	Champ ou action	Valeur
Project Name	Project name Project location Create project subdirectory	TD1 CoCiNum/vivado Oui
Project Type		RTL Project
Add Sources	Add Files Copy sources into project	CoCiNum/src/vhdl/td1/td_combinatoire.vhd Non
Add Constraints		
Default Part	Family Package Speed Part	Artix-7 cpg236 -1 xc7a35tcpg236-1

→ À l'étape `New Project Summary`, vérifiez les informations et pressez le bouton Finish.

Le fichier **td_combinatoire.vhd** n'est pas complet. Dans la suite on vous demandera de le modifier / compléter au fur et à mesure du TD.

1 Entité du circuit

Pour rappel *l'entité* est l'une des deux parties nécessaires (avec *l'architecture*) à la description d'un composant (ou circuit logique). L'entité définit les entrées et sorties de l'architecture (vue extérieure).

1.1 Analyse des entrées et sorties

La Figure 2 représente la vue extérieure de notre système.

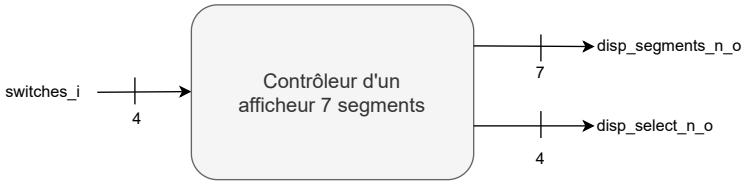

FIGURE 2 – Vue extérieure du contrôleur de l'afficheur 7 segments

— `switches_i` est un signal sur 4 bits qui correspond à l'état des 4 interrupteurs utilisées pour indiquer la valeur à afficher.
— `disp_segments_n_o` est un signal sur 7 bits qui correspond aux segments de l'afficheur 7 segments. Le signal `disp_segments_n_o(0)` correspond au segment g et `disp_segments_n_o(6)` correspond au segment a.
— `disp_select_n_o` est un signal sur 4 bits qui permet d'indiquer quel afficheur utiliser. La sélection d'un afficheur se fait par la mise à '0' du bit correspondant.

1.2 Rappel (ou pas)

On rappelle la syntaxe générale d'une *entité* :

```
entity Nom_Entite is
  port(
    signal1 : in type_signal1;   -- Signal d'entrée
    signal2 : in type_signal2;   -- Signal d'entrée
    signal3 : out type_signal3  -- Signal de sortie
  );
end entity Nom_Entite;
```

Et les deux principaux `type`s sont :
— `std_logic` pour définir un signal sur 1 bit.
— `std_logic_vector(n downto 0)` pour définir un signal sur n+1 bits.

1.3 Travail à faire

À partir de la Figure 2 :
→ Déterminer les entrées et sorties du système ainsi que leur type.
→ Compléter l'entité du fichier `td_combinatoire.vhd`.

2 Synthèse du circuit

La synthèse de la description matérielle en un circuit électronique se fait en plusieurs étapes :

1. Synthèse logique
2. Placement
3. Routage

À la fin de ces étapes, un fichier de configuration, appelé `bitstream` est généré. Ce fichier sera ensuite utilisé pour « programmé » le FPGA.

2.1 Génération du `bitstream`

→ Générer le `bitstream` (`Flow Navigator → Program and Debug → Generate Bitstream`).

Attention : À ce stade le *pop-up* suivant doit s'afficher :

Bitstream Generation failed

Si l'on regarde dans la console des messages (`write bitstream → DRC → Pin Planning`) on a l'erreur suivante :

Unspecified I/O Standard

Ce message indique que différents signaux de l'entité ne sont pas affectés à des broches du FPGA. Pour cela il faut ajouter un fichier de contrainte à votre projet.

→ Lire le fichier `BASYS3_td_combinatoire.xdc` et l'ajouter à votre projet (`File → Add sources → Add or created constraints`)

→ Générer le `bitstream` (pour de bon) puis le charger dans le FPGA :
- À la fin de l'opération, la boîte de dialogue `Bitstream Generation Completed` s'affiche. Choisissez `Open Hardware Manager`.
- Vérifiez que l'interrupteur d'alimentation de votre carte Basys3 est en position OFF. Le cavalier situé à côté de l'interrupteur doit être en position USB.
- Reliez le connecteur micro-USB de la carte à un port USB de votre PC. Mettez la carte sous tension.
- En haut du panneau `Hardware Manager`, pressez `Open target` et choisissez `Auto Connect`.
- Pressez ensuite Program Device.

→ Vérifier le fonctionnement du circuit. Que peut-on constater ?

3 Architecture du circuit

En `VHDL` l'*architecture* spécifie comment les signaux internes sont traités à l'intérieur du composant pour produire le comportement désiré.

La syntaxe est :

```
architecture Nom_Architecture of Nom_Entite is
  -- (Partie déclarative) Déclarations locales ici
begin
  -- (Partie comportementale ou structurelle) Logique ici
end architecture Nom_Architecture;
```

Pour décrire un circuit combinatoire, il existe différent types d'affectation :
- Affectation simple
- Affectation conditionnelle
- Affectation concurrente sélective

La syntaxe est présentée en Annexe 2.

3.1 Affectation simple

Les lignes *29* à *60* du fichier `td_combinatoire.vhd` correspondent aux affectations des différents segments avec des affectations simples.

→ Compléter l'affectation du dernier segment (lignes *57* à *59*) en s'aidant de l'Annexe 1.
→ Vérifier le fonctionnement du système.

Lorsque que l'on synthétise un circuit sur un FPGA , il est intéressant de regarder les statistiques d'utilisation. Les informations qui nous intéressent sont :
— Le nombre de bascules D utilisées (*FF*).
— Le nombre de LUT utilisées.
— Le nombre de broches d'entrées/sorties utilisées (*IO*).

→ Relever les informations sur les statistiques d'utilisation :
  — Dans la barre d'outils en haut de la fenêtre, pressez le bouton `Project Summary`, dont l'icône est une lettre Sigma (Σ).

→ Mettre en commentaire le code correspondant aux affectations simples (*sélectionner le code à commenter, puis « clic droit », toggle comment*).

3.2 Affectation conditionnelle

Les lignes *63* à *80* du fichier `td_combinatoire.vhd` correspondent aux affectations des différents segments avec des affectations conditionnelles.

→ Compléter les `???`.
→ Vérifier le fonctionnement du système.
→ Relever les informations sur les statistiques d'utilisation

→ Mettre en commentaire le code correspondant aux affectations conditionnelles.

3.3 Affectation concurrente sélective

Les lignes *83* à *101* du fichier `td_combinatoire.vhd` correspondent aux affectations des différents segments avec des affectations concurrentes sélectives.

→ Compléter les `???`.
→ Vérifier le fonctionnement du système.
→ Relever les informations sur les statistiques d'utilisation

3.4 Bilan sur les statistiques d'utilisation

→ Comparer les différents relevés.
→ Que peut-on en conclure ?

4 Première amélioration : détection de la parité et des nombres premiers

On veut maintenant pouvoir allumer deux leds afin d'indiquer :
— Le nombre à afficher est pair : `led_parite_o`.
— Le nombre à afficher est un nombre premier : `led_premier_o`.

Pour savoir si un nombre est pair en binaire, il suffit de regarder la valeur du bit de poids faible.

Pour savoir si un nombre est premier, on part du principe que l'on connaît la liste des nombres premiers entre 0 et 15 : 2, 3, 5, 7, 11 et 13.

4.1 Modification du code vhdl

→ Modifier l'entité pour ajouter les deux nouveaux signaux de sortie.
→ Ajouter dans l'architecture les affectations pour ces deux signaux de sortie.

4.2 Affectation des broches

→ Ajouter au ficher des contraintes les affectations relatives aux 2 leds :

```
set_property PACKAGE_PIN P1  [get_ports {led_parite_o}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_parite_o}]

set_property PACKAGE_PIN L1  [get_ports {led_premier_o}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_premier_o}]
```

4.3 Simulation du circuit

→ Démarrez le simulateur : `Flow Navigator` → `Simulation` → `Run Simulation` → `Run Behavioral Simulation`.
→ Dans l'onglet `Tcl Console` écrire les 3 commandes suivantes :

```
restart

add_force -radix dec switches_i 0 0ns 1 5ns 2 10ns 3 15ns 4 20ns
5 25ns 6 30ns 7 35ns 8 40ns 9 45ns 10 50ns 11 55ns 12 60ns
13 65ns 14 70ns 15 75ns

run 80ns
```

→ Observer la cohérence des signaux `led_parite_o` et `led_premier_o`.

5 Seconde amélioration : filtrage des sorties

On veut maintenant pouvoir activer l'affichage de la valeur en fonction de l'état de deux interrupteurs correspondant aux signaux d'entrées `switch_parite_i` et `switch_premier_i`.

Voici le comportement attendu en fonction de l'état de ces signaux :

`switch_parite_i`	`switch_premier_i`	Description
0	0	Affichage de tous les nombres
0	1	Affichage des nombres premiers
1	0	Affichage des nombres pairs
1	1	Affichage des nombres pairs ET premiers

Au niveau de code il sera nécessaire de déclarer trois signaux suivants :

```
signal s_disp_segments_n : std_logic_vector(6 downto 0);
signal s_led_parite : std_logic;
signal s_led_premier : std_logic;
```

Pour la partie affectation des broches, il faut ajouter les lignes suivantes au fichier de contraintes :

```
set_property PACKAGE_PIN T1  [get_ports {switch_parite_i}]
set_property IOSTANDARD LVCMOS33 [get_ports {switch_parite_i}]

set_property PACKAGE_PIN R2  [get_ports {switch_premier_i}]
set_property IOSTANDARD LVCMOS33 [get_ports {switch_premier_i}]
```

→ Modifier votre circuit afin d'intégrer les nouvelles exigences.

Annexe 1 : Table de vérité du décodeur pour l'afficheur 7 segments

Comme présenté sur la Figure 3, un afficheur 7 segments est un composant permettant d'afficher un chiffre à l'aide de 7 LEDs. Chacune des LEDs est associée à un segment. Un afficheur 7 segments se compose de 7 entrées qui vont permettre d'indiquer le segment à allumer.

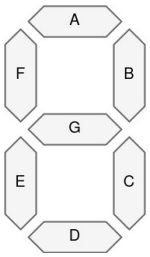

FIGURE 3 – Afficheur 7 segments

Sur la carte d'évaluation `BASYS3` l'allumage d'un segment se fait par la mise à '0' de la broche correspondante.

Ainsi la table de vérité est :

Entrées	**Hex**	**a**	**b**	**c**	**d**	**e**	**f**	**g**
0000	0	0	0	0	0	0	0	1
0001	1	1	0	0	1	1	1	1
0010	2	0	0	1	0	0	1	0
0011	3	0	0	0	0	1	1	0
0100	4	1	0	0	1	1	0	0
0101	5	0	1	0	0	1	0	0
0110	6	0	1	0	0	0	0	0
0111	7	0	0	0	1	1	1	1
1000	8	0	0	0	0	0	0	0
1001	9	0	0	0	0	1	0	0
1010	A	0	0	0	1	0	0	0
1011	B	1	1	0	0	0	0	0
1100	C	0	1	1	0	0	0	1
1101	D	1	0	0	0	0	1	0
1110	E	0	1	1	0	0	0	0
1111	F	0	1	1	1	0	0	0

Annexe 2 : Affectation pour la description d'un circuit combinatoire

Pour illustrer les différents types d'affectation, prenons l'exemple de la porte logique `ou exclusif` où :

— A et B sont les entrées
— Y est la sortie

Affectation simple

```
architecture Simple of Porte_OR_EX is
begin
  Y <= A xor B;
end architecture Simple;
```

Affectation conditionnelle

```
architecture Conditionnelle of Porte_OR_EX is
begin
  Y <= '1' when ((A='1' and B='0') or (A='0' and B='1')) else
       '0';
end architecture Simple;
```

Affectation concurrente sélective

```
architecture Selective of Porte_OR_EX is

signal SEL : std_logic_vector(1 downto 0);

begin

  SEL <= A & B;           --Concatenation des signaux A et B

  with SEL select
    Y <= '0' when "00",   -- Si SEL vaut "00"
         '1' when "01",   -- Si SEL vaut "01"
         '1' when "10",   -- Si SEL vaut "10"
         '0' when others; -- Valeur par défaut (SEL vaut "11")
end architecture Simple;
```